01 · Quickstart

Siddharth M. Bhatia · siddharth@sidmb.com
July 1, 2026

Contents

What just happened 2
Two *different* models 2
One-off generation without committing 2

The fastest path: `Conversation.from_models` scaffolds a two-party conversation from a tuple of models — each an HF id or an already-loaded model (`ModelLike`). If two ids are identical, the weights are loaded **once** and shared between the two speakers.

```python
from interlens import Conversation

# Two speakers backed by the same 0.5B model (one weight load, shared).
conv = Conversation.from_models(
    ("Qwen/Qwen2.5-0.5B-Instruct", "Qwen/Qwen2.5-0.5B-Instruct"),
    names=("alice", "bob"),
    device="cuda",          # "cpu" / "mps" also work for a smoke test
    temperature=0.8,        # **gen_kwargs are forwarded to both participants
    max_new_tokens=128,
    shared_context="Let's debate: is cereal a soup?",   # opening framing (see below)
)
conv.run(turns=4, first="alice")

print(conv.transcript)  # [i] author: content  (also conv.transcript.pretty())
print(conv.transcript.pretty(metadata=True))    # include per-turn metadata (reasoning, tool trail, token counts)

# See exactly what one model is conditioned on — role-swapped to its POV, WITH chat-template special tokens:
print(conv.render_templated(pov="alice", tokenize=False))   # `pov=` takes a name/index/Participant
```

What just happened

- **`shared_context=...`** seeds the opening without touching the transcript: it's a neutral, **moderator**-voiced turn everyone sees (scenario/topic framing). Pair it with **`shared_system_prompt=...`** for system-role instructions.
- **`prompt=...`** (on `from_models` and `run`) is the alternative when the opener should read as something a *speaker* said: a `str` is attributed to the **last** participant (so the `first` speaker replies to it), a `Message` sets the author explicitly. Use `shared_context` for neutral framing, `prompt` for a participant-voiced line.
- **`conv.run(turns=4, first="alice")`** alternates speakers for 4 turns starting with alice. `first` accepts a **name** (`"alice"`), an **index** (`0`), or a **`Participant`** object. `run` requires at least one of `turns=` or `until=` (a stop condition).
- **`conv.transcript`** is the shared state — a list of `Message`s (`.author`, `.content`, `.metadata`). You can still append to it directly for finer control.
- **`conv.participant("alice")`** looks a participant up by name.

Two *different* models

```
conv = Conversation.from_models(("Qwen/Qwen2.5-3B-Instruct", "google/
     gemma-2-2b-it"), names=("qwen", "gemma"), device="cuda")
```

Each id resolves to its family-correct participant class automatically (Qwen vs Gemma chat templates, tool formats, system-role handling) from its `config.model_type` — see 03.

One-off generation without committing

To sample a reply **without** mutating the transcript:

```
message = conv.sample("alice", "Quick: name a color.")
print(message.content)
```

Next: building conversations by hand for per-speaker system prompts, moderators, and stop conditions.

© 2026 Siddharth M. Bhatia. All rights reserved. Interlens is licensed under the GNU Affero General Public License v3.0.